```
const STORAGE_KEY = "inkwell_posts_v1";
const LIKES_KEY = "inkwell_likes_v1";

function loadPosts() {
  const raw = localStorage.getItem(STORAGE_KEY);
  if (raw) {
    try { return JSON.parse(raw); } catch (e) { /* fall through */ }
  }
  localStorage.setItem(STORAGE_KEY, JSON.stringify(SEED_POSTS));
  return [...SEED_POSTS];
}
function savePosts(posts) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(posts));
}
function loadLikedSet() {
  const raw = localStorage.getItem(LIKES_KEY);
  try { return new Set(JSON.parse(raw) || []); } catch (e) { return new Set(); }
}
function saveLikedSet(set) {
  localStorage.setItem(LIKES_KEY, JSON.stringify([...set]));
}

let posts = loadPosts();
let likedSet = loadLikedSet();
let editingPostId = null;
let activeCategory = "all";
```

```
function uid() { return "p" + Math.random().toString(36).slice(2, 9); }

function fmtDate(iso) {
  const d = new Date(iso + "T00:00:00");
  return d.toLocaleDateString("en-US", { month: "short", day: "numeric", year: "numeric" });
}

function initials(name) {
  return name.split(" ").map(w => w[0]).join("").slice(0, 2).toUpperCase();
}

function catLabel(id) {
  const c = CATEGORIES.find(c => c.id === id);
  return c ? c.label : id;
}

function estimateReadTime(html) {
  const text = html.replace(/<[^>]+>/g, " ");
  const words = text.trim().split(/\s+/).filter(Boolean).length;
  return Math.max(1, Math.round(words / 200));
}

function plainExcerpt(html, len = 140) {
  const text = html.replace(/<[^>]+>/g, " ").replace(/\s+/g, " ").trim();
  return text.length > len ? text.slice(0, len).trim() + "…" : text;
}

function toast(msg) {
```

```
  const el = document.getElementById("toast");
  el.textContent = msg;
  el.classList.add("show");
  clearTimeout(toast._t);
  toast._t = setTimeout(() => el.classList.remove("show"), 2400);
}

function catIcon(name, size = 20) {
  const icons = {
    code: `<path d="M7 5L2 10l5 5M13 5l5 5-5 5" stroke="currentColor" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round" fill="none"/>`,
    pen: `<path d="M4 16l1-4L14 3l3 3-9 9-4 1z" stroke="currentColor" stroke-width="1.5" stroke-linejoin="round" fill="none"/>`,
    compass: `<circle cx="10" cy="10" r="7.5" stroke="currentColor" stroke-width="1.5" fill="none"/><path d="M13 7l-2 5-4 1 2-5 4-1z" stroke="currentColor" stroke-width="1.3" stroke-linejoin="round" fill="none"/>`,
    leaf: `<path d="M5 15c-1-6 4-11 11-11 1 7-4 11-11 11z" stroke="currentColor" stroke-width="1.5" stroke-linejoin="round" fill="none"/><path d="M5 15l6-6" stroke="currentColor" stroke-width="1.5" stroke-linecap="round"/>`,
    cup: `<path d="M4 4h10v6a5 5 0 01-10 0V4z" stroke="currentColor" stroke-width="1.5" stroke-linejoin="round" fill="none"/><path d="M14 6h1.5a2 2 0 010 4H14" stroke="currentColor" stroke-width="1.5" fill="none"/>`,
    book: `<path d="M4 4.5h5a2 2 0 012 2v9a2 2 0 00-2-1.5H4V4.5zM16 4.5h-5a2 2 0 00-2 2" stroke="currentColor" stroke-width="1.5" stroke-linejoin="round" fill="none"/>`,
  };
  return `<svg width="${size}" height="${size}" viewBox="0 0 20 20" fill="none">${icons[name] || icons.book}</svg>`;
}

function navigate(route, param) {
  window.scrollTo({ top: 0, behavior: "instant" });
```

```
  closeMobileMenu();
  closeSearch();
  render(route, param);
  document.querySelectorAll(".nav-links a").forEach(a => {
    a.classList.toggle("active", a.dataset.route === route);
  });
}

document.addEventListener("click", (e) => {
  const link = e.target.closest("[data-route]");
  if (link) {
    e.preventDefault();
    navigate(link.dataset.route, link.dataset.param);
  }
});

function renderHome() {
  const app = document.getElementById("app");
  const published = posts.filter(p => p.status === "published");
  const featured = published.find(p => p.featured) || published[0];
  const rest = published.filter(p => p.id !== (featured && featured.id));
  const drafts = posts.filter(p => p.status === "draft");

  const catPills = ["all", ...CATEGORIES.map(c => c.id)].map(id => {
    const label = id === "all" ? "All stories" : catLabel(id);
    return `<button class="cat-pill ${activeCategory === id ? "active" : ""}" data-
cat="${id}">${label}</button>`;
  }).join("");
```

```
app.innerHTML = `
  <section class="hero page-enter">
    <div class="hero-grid">
      <div>
        <div class="hero-eyebrow">CMS-style blog platform · final project</div>
        <h1 class="hero-title">Words worth<br/><em>slowing down</em> for.</h1>
        <p class="hero-desc">Inkwell is a small publishing platform — write, edit, organize by category, and read distraction-free. Built to demonstrate a full content lifecycle, end to end.</p>
      </div>
      <div class="hero-side">
        <div class="hero-stats">
          <div>
            <div class="hero-stat-num">${published.length}</div>
            <div class="hero-stat-label">Published stories</div>
          </div>
          <div>
            <div class="hero-stat-num">${CATEGORIES.length}</div>
            <div class="hero-stat-label">Categories</div>
          </div>
          <div>
            <div class="hero-stat-num">${drafts.length}</div>
            <div class="hero-stat-label">Drafts in progress</div>
          </div>
        </div>
      </div>
    </div>
  </section>
```

```
  <div class="wrap">
    <div class="cat-row" id="catRow">${catPills}</div>
  </div>

  ${featured ? `
  <section class="featured">
    <div class="wrap">
      <div class="featured-card" data-route="article" data-param="${featured.id}">
        <div class="featured-img">
          <img src="${featured.cover}" alt="${featured.title}" />
          <span class="featured-tag">Featured</span>
        </div>
        <div>
          <div class="featured-meta">${catLabel(featured.category)}<span
class="dot"></span>${fmtDate(featured.date)}<span
class="dot"></span>${featured.readTime} min read</div>
          <h2 class="featured-title">${featured.title}</h2>
          <p class="featured-excerpt">${featured.excerpt}</p>
          <div class="featured-author">
            <div class="avatar">${initials(featured.author)}</div>
            <div>
              <div class="author-name">${featured.author}</div>
              <div class="author-role">${featured.authorRole}</div>
            </div>
          </div>
        </div>
      </div>
    </div>
```

```
    </section>` : ""}

    <section class="posts-section">
      <div class="wrap">
        <div class="section-head">
          <h2 class="section-title">${activeCategory === "all" ? "Latest stories" :
catLabel(activeCategory)}</h2>
          <span class="section-count" id="postCount"></span>
        </div>
        <div id="postGrid"></div>
      </div>
    </section>
  `;

  document.getElementById("catRow").addEventListener("click", e => {
    const btn = e.target.closest(".cat-pill");
    if (!btn) return;
    activeCategory = btn.dataset.cat;
    renderHome();
  });

  renderPostGrid();
}

function renderPostGrid() {
  const grid = document.getElementById("postGrid");
  const countEl = document.getElementById("postCount");
  if (!grid) return;
```

```
  let list = posts.filter(p => p.status === "published");
  const featuredId = (list.find(p => p.featured) || list[0] || {}).id;
  if (activeCategory === "all") {
    list = list.filter(p => p.id !== featuredId);
  } else {
    list = list.filter(p => p.category === activeCategory);
  }
  list = list.sort((a, b) => new Date(b.date) - new Date(a.date));

  countEl.textContent = `${list.length} ${list.length === 1 ? "story" : "stories"}`;

  if (list.length === 0) {
    grid.innerHTML = `
      <div class="empty-state">
        <svg width="48" height="48" viewBox="0 0 48 48" fill="none"><path d="M12 8h24v32H12z" stroke="currentColor" stroke-width="1.5"/><path d="M18 18h12M18 24h12M18 30h7" stroke="currentColor" stroke-width="1.5" stroke-linecap="round"/></svg>
        <h3>Nothing here yet</h3>
        <p>No published stories in this category.</p>
        <button class="btn-write" style="margin:0 auto" data-route="editor">Write the first one</button>
      </div>`;
    return;
  }

  grid.className = "post-grid";
  grid.innerHTML = list.map(p => `
    <article class="post-card" data-route="article" data-param="${p.id}">
```

```
    <div class="post-thumb"><img src="${p.cover}" alt="${p.title}" loading="lazy" /></div>
    <span class="post-cat">${catLabel(p.category)}</span>
    <h3 class="post-title">${p.title}</h3>
    <p class="post-excerpt">${p.excerpt}</p>
    <div class="post-foot">${p.author}<span class="dot"></span>${fmtDate(p.date)}<span class="dot"></span>${p.readTime} min</div>
   </article>
  `).join("");
}

function renderCategories() {
  const app = document.getElementById("app");
  const published = posts.filter(p => p.status === "published");

  app.innerHTML = `
   <section class="hero page-enter" style="padding-bottom:2.5rem">
    <div class="wrap">
     <div class="hero-eyebrow">Browse</div>
     <h1 class="hero-title" style="font-size:clamp(2.2rem,4.5vw,3.6rem)">Find your <em>next read</em></h1>
     <p class="hero-desc">Stories organized by topic — from technology to slow travel.</p>
    </div>
   </section>
   <div class="wrap">
    <div class="cat-page-grid">
     ${CATEGORIES.map(c => {
      const count = published.filter(p => p.category === c.id).length;
      return `
      <div class="cat-card" data-route="category-detail" data-param="${c.id}">
```

```
          <div class="cat-card-icon">${catIcon(c.icon, 22)}</div>
          <div class="cat-card-title">${c.label}</div>
          <div class="cat-card-count">${count} ${count === 1 ? "story" : "stories"}</div>
        </div>`;
      }).join("")}
    </div>
  </div>
 `;
}

function renderCategoryDetail(catId) {
 activeCategory = catId;
 navigate("home");
}

function renderArticle(postId) {
 const app = document.getElementById("app");
 const post = posts.find(p => p.id === postId);
 if (!post) {
  app.innerHTML = `<div class="wrap"><div class="empty-state"><h3>Story not found</h3><p>It may have been removed.</p><button class="btn-write" style="margin:0 auto" data-route="home">Back home</button></div></div>`;
  return;
 }

 const related = posts
  .filter(p => p.status === "published" && p.category === post.category && p.id !== post.id)
  .slice(0, 3);
```

```
  const isLiked = likedSet.has(post.id);


  app.innerHTML = `
    <article class="article-hero page-enter">
      <div class="wrap">
        <span class="back-link" data-route="home">
          <svg width="14" height="14" viewBox="0 0 14 14" fill="none"><path d="M9 2L3 7l6 5" stroke="currentColor" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round"/></svg>
          Back to stories
        </span>
        <div class="article-meta-top">
          ${post.status === "draft" ? '<span style="color:var(--rust)">Draft</span><span>·</span>' : ""}
          ${catLabel(post.category)}
        </div>
        <h1 class="article-title">${post.title}</h1>
        ${post.sub ? `<p class="article-sub">${post.sub}</p>` : ""}
        <div class="article-byline">
          <div class="byline-left">
            <div class="avatar">${initials(post.author)}</div>
            <div>
              <div class="author-name">${post.author}</div>
              <div class="author-role">${post.authorRole} · ${fmtDate(post.date)} · ${post.readTime} min read</div>
            </div>
          </div>
          <div class="byline-right">
            <button class="action-btn" data-route="editor" data-param="${post.id}">
```

<svg width="14" height="14" viewBox="0 0 14 14" fill="none"><path d="M9.5 1.5l3 3L4 13H1v-3l8.5-8.5z" stroke="currentColor" stroke-width="1.3" stroke-linejoin="round"/></svg>

Edit

</button>

</article>

${post.body}

<button class="action-btn like ${isLiked ? "active" : ""}" id="likeBtn">

<svg width="14" height="14" viewBox="0 0 14 14" fill="${isLiked ? "currentColor" : "none"}"><path d="M7 12.5S1.5 9 1.5 5.2A2.7 2.7 0 017 3.5a2.7 2.7 0 015.5 1.7C12.5 9 7 12.5 7 12.5z" stroke="currentColor" stroke-width="1.3" stroke-linejoin="round"/></svg>

${post.likes}

</button>

<button class="action-btn danger" id="deleteBtn">

<svg width="14" height="14" viewBox="0 0 14 14" fill="none"><path d="M2.5 4h9M5.5 4V2.5h3V4M3.5 4l.5 8h6l.5-8" stroke="currentColor" stroke-width="1.3" stroke-linecap="round" stroke-linejoin="round"/></svg>

Delete story

</button>

${post.status === "draft" ? `<button class="action-btn" id="publishBtn" style="color:var(--forest);border-color:var(--forest)">Publish now</button>` : ""}

${related.length ? `

```
    <div class="section-head"><h2 class="section-title">More in ${catLabel(post.category)}</h2></div>
    <div class="post-grid">
      ${related.map(p => `
        <article class="post-card" data-route="article" data-param="${p.id}">
          <div class="post-thumb"><img src="${p.cover}" alt="${p.title}" loading="lazy" /></div>
          <span class="post-cat">${catLabel(p.category)}</span>
          <h3 class="post-title">${p.title}</h3>
          <p class="post-excerpt">${p.excerpt}</p>
          <div class="post-foot">${p.author}<span class="dot"></span>${fmtDate(p.date)}</div>
        </article>
      `).join("")}
    </div>
  </div>` : ""}
`;

document.getElementById("likeBtn").addEventListener("click", () => {
  if (likedSet.has(post.id)) {
    likedSet.delete(post.id);
    post.likes = Math.max(0, post.likes - 1);
  } else {
    likedSet.add(post.id);
    post.likes += 1;
  }
  saveLikedSet(likedSet);
  savePosts(posts);
  renderArticle(postId);
```

```
  });

  document.getElementById("deleteBtn").addEventListener("click", () => {
    if (confirm(`Delete "${post.title}"? This can't be undone.`)) {
      posts = posts.filter(p => p.id !== post.id);
      savePosts(posts);
      toast("Story deleted");
      navigate("home");
    }
  });

  const publishBtn = document.getElementById("publishBtn");
  if (publishBtn) {
    publishBtn.addEventListener("click", () => {
      post.status = "published";
      savePosts(posts);
      toast("Story published");
      renderArticle(postId);
    });
  }
}

function renderEditor(postId) {
  const app = document.getElementById("app");
  editingPostId = postId || null;
  const existing = editingPostId ? posts.find(p => p.id === editingPostId) : null;

  const draft = existing ? { ...existing } : {
```

```
    id: uid(), title: "", sub: "", body: "", category: CATEGORIES[0].id,
    author: "Shubham", authorRole: "Contributing Writer",
    cover: COVER_OPTIONS[0], status: "draft", date: new Date().toISOString().slice(0, 10),
    readTime: 1, likes: 0, featured: false,
  };

  app.innerHTML = `
    <div class="editor-wrap page-enter">
      <div class="editor-head">
        <span class="editor-eyebrow">${existing ? "Editing story" : "New story"}</span>
        <div class="editor-actions">
          <button class="btn-ghost-sm" id="saveDraftBtn">Save draft</button>
          <button class="btn-publish" id="publishStoryBtn">Publish</button>
        </div>
      </div>

      <div class="field">
        <input type="text" id="titleInput" placeholder="Your story title…" value="${draft.title}"
/>
        <input type="text" id="subInput" placeholder="Add a one-line subtitle (optional)"
value="${draft.sub || ""}" />
      </div>

      <div class="editor-meta-row">
        <div>
          <span class="field-label">Category</span>
          <select id="catSelect">
            ${CATEGORIES.map(c => `<option value="${c.id}" ${draft.category === c.id ?
"selected" : ""}>${c.label}</option>`).join("")}
```

```
        </select>
      </div>
      <div>
        <span class="field-label">Author</span>
        <input type="text" class="text-input" id="authorInput" value="${draft.author}" />
      </div>
      <div>
        <span class="field-label">Author role</span>
        <input type="text" class="text-input" id="roleInput" value="${draft.authorRole}" />
      </div>
    </div>

    <div class="field">
      <span class="field-label">Cover image</span>
      <div class="cover-picker" id="coverPicker">
        ${COVER_OPTIONS.map((url, i) => `
          <div class="cover-opt ${draft.cover === url ? "active" : ""}" data-cover="${url}">
            <img src="${url}" alt="Cover option ${i + 1}" />
          </div>`).join("")}
      </div>
    </div>

    <div class="field">
      <span class="field-label">Story body</span>
      <div class="toolbar">
        <button type="button" class="tb-btn" data-cmd="bold" title="Bold">B</button>
        <button type="button" class="tb-btn" data-cmd="italic" title="Italic" style="font-
style:italic">i</button>
```

```
        <span class="tb-sep"></span>
        <button type="button" class="tb-btn" data-cmd="h2" title="Heading">H2</button>
        <button type="button" class="tb-btn" data-cmd="quote" title="Quote">"</button>
        <span class="tb-sep"></span>
        <button type="button" class="tb-btn" data-cmd="ul" title="Bullet list">•</button>
      </div>
      <textarea id="bodyInput" placeholder="Write your story. Select text and use the toolbar, or write plain paragraphs separated by blank lines.">${draft.bodyRaw || (existing ? htmlToRaw(draft.body) : "")}</textarea>
    </div>

    <div class="editor-foot">
      <span id="wordCountLabel">0 words</span>
      <span>Autosaves as draft on save</span>
    </div>
  </div>
`;

let selectedCover = draft.cover;
document.getElementById("coverPicker").addEventListener("click", e => {
  const opt = e.target.closest(".cover-opt");
  if (!opt) return;
  selectedCover = opt.dataset.cover;
  document.querySelectorAll(".cover-opt").forEach(o => o.classList.remove("active"));
  opt.classList.add("active");
});

const bodyInput = document.getElementById("bodyInput");
const wordLabel = document.getElementById("wordCountLabel");
```

```
  function updateWordCount() {
    const words = bodyInput.value.trim().split(/\s+/).filter(Boolean).length;
    wordLabel.textContent = `${words} word${words === 1 ? "" : "s"} · ~${Math.max(1, Math.round(words / 200))} min read`;
  }
  bodyInput.addEventListener("input", updateWordCount);
  updateWordCount();

  document.querySelectorAll(".tb-btn").forEach(btn => {
    btn.addEventListener("click", () => applyToolbarCmd(btn.dataset.cmd, bodyInput));
  });

  function collectForm(status) {
    const title = document.getElementById("titleInput").value.trim();
    const bodyRaw = bodyInput.value.trim();
    if (!title) { toast("Give your story a title first"); return null; }
    if (!bodyRaw) { toast("Your story needs some words"); return null; }
    const bodyHtml = rawToHtml(bodyRaw);
    return {
      ...draft,
      title,
      sub: document.getElementById("subInput").value.trim(),
      category: document.getElementById("catSelect").value,
      author: document.getElementById("authorInput").value.trim() || "Anonymous",
      authorRole: document.getElementById("roleInput").value.trim() || "Contributor",
      cover: selectedCover,
      body: bodyHtml,
      bodyRaw,
```

```
    excerpt: plainExcerpt(bodyHtml),
    readTime: estimateReadTime(bodyHtml),
    status,
    date: existing ? draft.date : new Date().toISOString().slice(0, 10),
  };
 }

 document.getElementById("saveDraftBtn").addEventListener("click", () => {
  const result = collectForm("draft");
  if (!result) return;
  upsertPost(result);
  toast("Saved as draft");
  navigate("article", result.id);
 });

 document.getElementById("publishStoryBtn").addEventListener("click", () => {
  const result = collectForm("published");
  if (!result) return;
  upsertPost(result);
  toast("Story published");
  navigate("article", result.id);
 });
}

function upsertPost(post) {
 const idx = posts.findIndex(p => p.id === post.id);
 if (idx >= 0) posts[idx] = post; else posts.unshift(post);
 savePosts(posts);
```

```
}

/* Simple raw-text <-> light markup helpers for the editor */
function rawToHtml(raw) {
  return raw.split(/\n{2,}/).map(block => {
    block = block.trim();
    if (!block) return "";
    if (block.startsWith("## ")) return `<h2>${block.slice(3)}</h2>`;
    if (block.startsWith("> ")) return `<blockquote>${block.slice(2)}</blockquote>`;
    if (block.startsWith("- ")) {
      const items = block.split("\n").map(l => `<li>${l.replace(/^- /, "")}</li>`).join("");
      return `<ul>${items}</ul>`;
    }
    let inline = block
      .replace(/\*\*(.+?)\*\*/g, "<strong>$1</strong>")
      .replace(/\*(.+?)\*/g, "<em>$1</em>");
    return `<p>${inline}</p>`;
  }).join("\n");
}
function htmlToRaw(html) {
  return html
    .replace(/<h2>(.*?)<\/h2>/g, "## $1\n\n")
    .replace(/<blockquote>(.*?)<\/blockquote>/g, "> $1\n\n")
    .replace(/<ul>(.*?)<\/ul>/gs, (m, inner) => inner.replace(/<li>(.*?)<\/li>/g, "- $1\n") + "\n")
    .replace(/<\/?strong>/g, "**")
    .replace(/<\/?em>/g, "*")
    .replace(/<p>(.*?)<\/p>/g, "$1\n\n")
    .trim();
```

```
}
function applyToolbarCmd(cmd, textarea) {
  const start = textarea.selectionStart, end = textarea.selectionEnd;
  const sel = textarea.value.slice(start, end) || "text";
  let wrapped;
  if (cmd === "bold") wrapped = `**${sel}**`;
  else if (cmd === "italic") wrapped = `*${sel}*`;
  else if (cmd === "h2") wrapped = `\n\n## ${sel}\n\n`;
  else if (cmd === "quote") wrapped = `\n\n> ${sel}\n\n`;
  else if (cmd === "ul") wrapped = `\n\n- ${sel}\n- item two\n\n`;
  textarea.setRangeText(wrapped, start, end, "end");
  textarea.focus();
  textarea.dispatchEvent(new Event("input"));
}

function renderAbout() {
  const app = document.getElementById("app");
  app.innerHTML = `
    <section class="about-hero page-enter">
      <div class="wrap">
        <h1>Built to demonstrate a real <em>content lifecycle.</em></h1>
        <p>Inkwell is a final web development project: a self-contained CMS-style blog platform covering creation, editing, organization, and reading — without a backend server.</p>
      </div>
    </section>
    <div class="wrap">
      <div class="about-grid">
        <div class="about-feature">
```

<svg width="20" height="20" viewBox="0 0 20 20" fill="none"><path d="M10.5 2.5l3 3L5 14H2v-3l8.5-8.5z" stroke="currentColor" stroke-width="1.4" stroke-linejoin="round"/></svg>

<h3>Write & edit</h3>

A distraction-free editor with a lightweight formatting toolbar, live word count, and reading-time estimate.

<svg width="20" height="20" viewBox="0 0 20 20" fill="none"><circle cx="9" cy="9" r="6" stroke="currentColor" stroke-width="1.4"/><path d="M17 17l-3.5-3.5" stroke="currentColor" stroke-width="1.4" stroke-linecap="round"/></svg>

<h3>Organize & find</h3>

Six categories, instant client-side search across titles, authors, and excerpts, and category landing pages.

<svg width="20" height="20" viewBox="0 0 20 20" fill="none"><path d="M3 10s2.5-5.5 7-5.5S17 10 17 10s-2.5 5.5-7 5.5S3 10 3 10z" stroke="currentColor" stroke-width="1.4"/><circle cx="10" cy="10" r="2" stroke="currentColor" stroke-width="1.4"/></svg>

<h3>Publish & manage</h3>

Draft and published states, likes, deletion with confirmation — the full lifecycle of a piece of content.

Under the hood

<h2 class="section-title" style="margin-bottom:.5rem">Stack & approach</h2>

```
      <p style="color:var(--ink-soft);max-width:600px">No frameworks, no build step — vanilla HTML, CSS and JavaScript with a small client-side router and <code>localStorage</code> standing in for a database, so the whole CMS lifecycle works without a backend.</p>
      <div class="tech-list">
        <span class="tech-chip">HTML5</span>
        <span class="tech-chip">CSS3 (Grid + Flexbox)</span>
        <span class="tech-chip">Vanilla JavaScript</span>
        <span class="tech-chip">localStorage persistence</span>
        <span class="tech-chip">Client-side routing</span>
        <span class="tech-chip">Responsive design</span>
      </div>
    </div>
  </div>
  `;
}

function render(route, param) {
  if (route === "home") renderHome();
  else if (route === "categories") renderCategories();
  else if (route === "category-detail") renderCategoryDetail(param);
  else if (route === "article") renderArticle(param);
  else if (route === "editor") renderEditor(param);
  else if (route === "about") renderAbout();
  else renderHome();
}

const searchPanel = document.getElementById("searchPanel");
const searchInput = document.getElementById("searchInput");
```

```
const searchResults = document.getElementById("searchResults");

function openSearch() {
  searchPanel.classList.add("open");
  setTimeout(() => searchInput.focus(), 200);
}
function closeSearch() {
  searchPanel.classList.remove("open");
  searchInput.value = "";
  searchResults.innerHTML = "";
}
document.getElementById("searchToggle").addEventListener("click", () => {
  searchPanel.classList.contains("open") ? closeSearch() : openSearch();
});
document.getElementById("searchClose").addEventListener("click", closeSearch);

searchInput.addEventListener("input", () => {
  const q = searchInput.value.trim().toLowerCase();
  if (!q) { searchResults.innerHTML = ""; return; }
  const results = posts.filter(p =>
    p.status === "published" && (
      p.title.toLowerCase().includes(q) ||
      p.author.toLowerCase().includes(q) ||
      p.excerpt.toLowerCase().includes(q) ||
      catLabel(p.category).toLowerCase().includes(q)
    )
  ).slice(0, 8);
```

```
  if (results.length === 0) {
    searchResults.innerHTML = `<div class="search-empty">No stories match "${searchInput.value}"</div>`;
    return;
  }
  searchResults.innerHTML = results.map(p => `
    <div class="search-result-item" data-route="article" data-param="${p.id}">
      <span class="sr-title">${p.title}</span>
      <span class="sr-cat">${catLabel(p.category)}</span>
    </div>
  `).join("");
});

document.addEventListener("keydown", e => {
  if ((e.metaKey || e.ctrlKey) && e.key === "k") { e.preventDefault(); openSearch(); }
  if (e.key === "Escape") closeSearch();
});

const hamburger = document.getElementById("hamburger");
const mobileMenu = document.getElementById("mobileMenu");
function closeMobileMenu() {
  hamburger.classList.remove("open");
  mobileMenu.classList.remove("open");
}
hamburger.addEventListener("click", () => {
  hamburger.classList.toggle("open");
  mobileMenu.classList.toggle("open");
});
```

```
navigate("home");

const CATEGORIES = [
  { id: "tech", label: "Technology", icon: "code" },
  { id: "design", label: "Design", icon: "pen" },
  { id: "travel", label: "Travel", icon: "compass" },
  { id: "life", label: "Life & Thought", icon: "leaf" },
  { id: "food", label: "Food", icon: "cup" },
  { id: "culture", label: "Culture", icon: "book" },
];

const COVER_OPTIONS = [
  "https://images.unsplash.com/photo-1455390582262-044cdead277a?w=900&q=80",
  "https://images.unsplash.com/photo-1457369804613-52c61a468e7d?w=900&q=80",
  "https://images.unsplash.com/photo-1432821596592-e2c18b78144f?w=900&q=80",
  "https://images.unsplash.com/photo-1517842645767-c639042777db?w=900&q=80",
  "https://images.unsplash.com/photo-1499750310107-5fef28a66643?w=900&q=80",
  "https://images.unsplash.com/photo-1418065460487-3e41a6c84dc5?w=900&q=80",
];

const SEED_POSTS = [
  {
    id: "p1",
    title: "The Quiet Discipline of Writing Every Day",
    sub: "Why consistency beats inspiration, and how a blank page stops being terrifying.",
    excerpt: "There's a particular kind of fear that comes from facing a blank page. Most writers never talk about it honestly — they talk around it.",
```

```
    body: `<p>There's a particular kind of fear that comes from facing a blank page. Most writers never talk about it honestly — they talk around it, dress it up as "writer's block," and wait for inspiration to rescue them.</p>

    <p>But inspiration is unreliable. It shows up on its own schedule, usually right after you've given up for the day. The writers who actually finish things — who ship books, articles, essays — rarely wait for it.</p>

    <h2>Showing up beats waiting</h2>

    <p>What separates a working writer from someone who dreams of writing is not talent. It's the willingness to sit down on a day when nothing feels worth saying, and write four hundred mediocre words anyway.</p>

    <blockquote>You can't edit a blank page. But you can edit a bad one.</blockquote>

    <p>This is the entire secret. Bad drafts are raw material. Blank pages are nothing. The discipline isn't about forcing brilliance on command — it's about producing something, anything, that future-you can shape into something worth reading.</p>

    <h2>Build a small, boring ritual</h2>

    <p>The same coffee. The same chair. The same fifteen minutes before the rest of the day intrudes. Rituals aren't superstition — they're a signal to your brain that it's time to work, removing the friction of deciding whether today is "a writing day."</p>

    <p>Eventually the question stops being whether you feel like writing. It becomes simply: what page am I on.</p>`,

    category: "life",

    author: "Maya Chen",

    authorRole: "Staff Writer",

    cover: "https://images.unsplash.com/photo-1455390582262-044cdead277a?w=900&q=80",

    status: "published",

    date: "2026-05-12",

    readTime: 4,

    likes: 128,

    featured: true,

  },

  {
```

id: "p2",

title: "Designing for Trust: Lessons from Fintech Interfaces",

sub: "Small interface decisions that make people feel safe handing over their money.",

excerpt: "Trust in software isn't built with a single hero shot. It's built in increments — through copy, through timing, through what you choose not to show.",

body: ` Trust in software isn't built with a single hero shot. It's built in increments — through copy, through timing, through what you choose not to show.

When we redesigned our payments flow, the biggest lift in conversion didn't come from a new color palette. It came from removing a confirmation step that felt redundant to us but felt reassuring to users.

<h2>Friction is sometimes the feature</h2>

Conventional wisdom says reduce friction everywhere. But in financial products, a small amount of friction at the right moment — a confirmation screen before an irreversible action — signals competence, not clumsiness.

<h2>Words carry more weight than color</h2>

We spent weeks debating button colors and almost no time on the copy beside them. That ordering was backwards. "Your payment is encrypted and reversible for 24 hours" does more for trust than any shade of blue.

Design systems are good at consistency. They are not automatically good at empathy. That part still requires sitting with real users and watching where their cursor hesitates. `,

category: "design",

author: "Rohan Mehta",

authorRole: "Product Designer",

cover: "https://images.unsplash.com/photo-1457369804613-52c61a468e7d?w=900&q=80",

status: "published",

date: "2026-05-08",

readTime: 6,

likes: 94,

featured: true,

```
  },
  {
    id: "p3",
    title: "What Static Site Generators Get Right",
    sub: "A practical case for boring technology in 2026.",
    excerpt: "Every few months a new framework promises to fix the problems of the last one. Meanwhile, static files keep quietly working.",
    body: `<p>Every few months a new framework promises to fix the problems of the last one. Meanwhile, static files keep quietly working — served from a CDN edge, costing almost nothing, breaking almost never.</p>
    <p>For content that doesn't need to be regenerated on every request, static generation remains one of the best trade-offs in web engineering: instant loads, trivial caching, and a deploy story that fits in one command.</p>
    <h2>When dynamic rendering actually pays for itself</h2>
    <p>Personalization, real-time data, and heavy interactivity are real reasons to reach for server rendering or client-side frameworks. But a surprising amount of the web doesn't need any of that — and pretending it does adds complexity for no user benefit.</p>
    <p>Pick boring technology where boring technology solves the problem. Save the complexity budget for the parts of the product that actually need it.</p>`,
    category: "tech",
    author: "Maya Chen",
    authorRole: "Staff Writer",
    cover: "https://images.unsplash.com/photo-1432821596592-e2c18b78144f?w=900&q=80",
    status: "published",
    date: "2026-04-29",
    readTime: 5,
    likes: 76,
    featured: false,
  },
  {
```

```
    id: "p4",
    title: "Three Weeks on the Konkan Coast, Slowly",
    sub: "A travelogue about doing almost nothing, on purpose.",
    excerpt: "We didn't plan an itinerary. We picked a coastline and let each day decide itself — fishing villages, empty beaches, and a lot of coconut water.",
    body: `<p>We didn't plan an itinerary. We picked a coastline and let each day decide itself — fishing villages, empty beaches, and a lot of coconut water.</p>
    <p>Most travel writing is about doing as much as possible. This is the opposite kind of story: about staying in one small town for four days because the chai stall owner remembered our order, and that felt like enough of a reason.</p>
    <h2>The places without a name on the map</h2>
    <p>Some of the best afternoons happened in villages too small for the map app to label. A fisherman taught us how to read tide patterns by the color of the water. None of it was efficient. All of it was worth it.</p>
    <p>If you're used to checklist travel, the Konkan coast asks you to unlearn it. Go slow, stay longer than feels productive, and let boredom do some of the work.</p>`,
    category: "travel",
    author: "Ishaan Verma",
    authorRole: "Contributing Writer",
    cover: "https://images.unsplash.com/photo-1517842645767-c639042777db?w=900&q=80",
    status: "published",
    date: "2026-04-21",
    readTime: 7,
    likes: 152,
    featured: false,
  },
  {
    id: "p5",
    title: "The Case for Cooking the Same Five Meals",
```

sub: "Why a small, mastered repertoire beats a Pinterest board of recipes you'll never make.",

excerpt: "Somewhere along the way, home cooking became a performance of variety. Five meals, made well and often, might be the better answer.",

body: ` Somewhere along the way, home cooking became a performance of variety. Recipe apps reward novelty. But there's a quieter, more useful skill: knowing five meals so well you barely think while making them.

<h2>Mastery over menu</h2>

A dal you've made eighty times is better than a recipe you've followed once. Repetition builds intuition — when to add salt by feel, how long is "long enough" for onions to actually caramelize.

Pick five dishes that cover your week. Learn them properly. The variety can come from what you pair them with, not from chasing a new recipe every night. `,

category: "food",

author: "Ananya Iyer",

authorRole: "Contributing Writer",

cover: "https://images.unsplash.com/photo-1499750310107-5fef28a66643?w=900&q=80",

status: "published",

date: "2026-04-15",

readTime: 4,

likes: 61,

featured: false,

},

{

id: "p6",

title: "Reading Slowly in a Skimming Culture",

sub: "On the disappearing art of finishing a book without checking your phone.",

excerpt: "We've gotten remarkably good at skimming and remarkably bad at sitting with a single paragraph long enough to actually think about it.",

```
    body: `<p>We've gotten remarkably good at skimming and remarkably bad at sitting with a single paragraph long enough to actually think about it.</p>

    <p>Slow reading isn't about reading less. It's about reading with your full attention parked in one place, instead of split across a feed running in the background of your mind.</p>

    <h2>A small experiment</h2>

    <p>Try one chapter, phone in another room. Notice how long it takes for your mind to start reaching for a phantom notification. That itch is the thing slow reading is training you out of.</p>`,

    category: "culture",

    author: "Rohan Mehta",

    authorRole: "Product Designer",

    cover: "https://images.unsplash.com/photo-1455390582262-044cdead277a?w=900&q=80",

    status: "published",

    date: "2026-04-02",

    readTime: 5,

    likes: 88,

    featured: false,

  },

  {

    id: "p7",

    title: "Notes on Building a Component Library Nobody Hates",

    sub: "Draft — figuring out the right framing for this one.",

    excerpt: "Most internal design systems end up either too rigid or too loose. There's a narrow path between the two that actually gets adopted.",

    body: `<p>Most internal design systems end up either too rigid or too loose. There's a narrow path between the two that actually gets adopted by the teams it's meant to serve.</p>

    <p>This one's still rough — more notes than essay right now.</p>`,

    category: "tech",
```

```
    author: "Maya Chen",
    authorRole: "Staff Writer",
    cover: "https://images.unsplash.com/photo-1432821596592-
e2c18b78144f?w=900&q=80",
    status: "draft",
    date: "2026-05-18",
    readTime: 3,
    likes: 0,
    featured: false,
  },
];
```